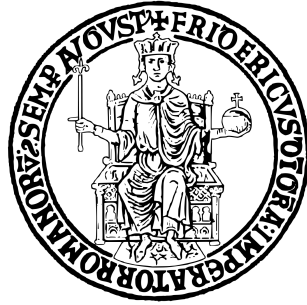




BugBoard26

Progetto di Ingegneria del Software – A.A. 2025/2026

Autori:

Simone Parente Martone - N86004297

Mario Penna - N86003308

Michela Pollio - N86003697

Dipartimento di Ingegneria Elettrica e delle Tecnologie
dell'Informazione (DIETI)
Università degli Studi di Napoli Federico II

Indice

1 Introduzione

Il presente documento descrive l'analisi e la specifica dei requisiti per il progetto *BugBoard26*, una piattaforma software per la gestione collaborativa di *issue* e *bug* durante il ciclo di sviluppo di un prodotto software.

L'obiettivo principale del sistema è fornire un ambiente:

- **semplice**, per facilitare la segnalazione e la gestione dei problemi;
- **sicuro**, per garantire l'integrità e la riservatezza dei dati;
- **efficiente**, per supportare le attività di classificazione, assegnazione e monitoraggio delle problematiche software.

Il sistema integra funzionalità avanzate come:

- filtri dinamici di ricerca,
- allegati di immagini,
- esportazione dei dati in diversi formati,
- generazione di report mensili sull'attività del team.

Il documento include:

- la specifica dettagliata dei requisiti software;
- la modellazione dei casi d'uso secondo il formalismo di *Alistair Cockburn* **cockburn-use-cases**;
- la descrizione dell'interazione tra utenti e sistema, supportata da diagrammi e mock-up.

Questo testo costituisce una guida completa per **sviluppatori**, **project manager** e **stakeholder**, fornendo una visione chiara e strutturata delle funzionalità e delle logiche alla base del progetto *BugBoard26*.

2 Specifica dei Requisiti Software

2.1 Attori del Sistema

Considerando il tipo di sistema in sviluppo — una piattaforma per la gestione collaborativa di progetti software — il background tecnico degli utenti sarà relativamente omogeneo. La maggior parte saranno sviluppatori, ad eccezione dei team leader, che potrebbero avere competenze tecniche meno avanzate.

Possiamo quindi definire due classi di utenti:

- **Utente standard:** sviluppatori software, tester e figure tecniche operative.
- **Utente amministratore:** team leader, manager, project manager e altri stakeholders con responsabilità di coordinamento e supervisione del progetto.

Tabella 2.1: Caratterizzazione degli attori

Attore	Utente Standard (Sviluppatore, Tester)	Utente Amministratore (Team Leader, Project Manager)
Ruoli e Responsabilità	Segnalazione issues, assegnazione task, sviluppo codice, esecuzione e supervisione dei test, documentazione tecnica, collaborazione su task assegnati	Coordinamento team, monitoraggio avanzamento, gestione priorità, reporting, supervisione generale del progetto
Competenze Tecniche	Conoscenza avanzata di linguaggi di programmazione, testing, version control, metodologie di sviluppo	Conoscenze tecniche di base/intermedie, competenze gestionali, pianificazione progetti, gestione risorse
Frequenza e Modalità d'Uso	Utilizzo quotidiano, accesso continuo durante orario lavorativo, interazione con task e repository	Utilizzo regolare (giornaliero/settimanale), focus su dashboard, reportistica e gestione team

Attore	Utente Standard (Sviluppatore, Tester)	Utente Amministratore (Team Leader, Project Manager)
Obiettivi	Completare task assegnati, collaborare con il team, mantenere qualità del codice, rispettare le deadline	Garantire rispetto timeline, allocare risorse efficacemente, monitorare performance del team, gestire rischi
Privilegi di Accesso	Creazione, lettura/scrittura su task, accesso ai repository del progetto, creazione issue e commenti	Accesso completo al progetto, creazione, modifica, eliminazione task, gestione utenti, configurazione progetto, accesso a statistiche e reportistica avanzata

2.2 Requisiti Non Funzionali

Il sistema sarà composto da due moduli principali: un **frontend** e un **backend**. Il frontend sarà una *Single Page Application (SPA)*, sviluppata presumibilmente tramite framework come **Angular** o **React**, mentre il backend sarà implementato in **Java**, utilizzando tecnologie come **Spring**, **Hibernate** o **Jakarta EE**.

2.2.1 Requisiti del Prodotto

Usabilità

- Il sistema dovrà essere facile da usare ed intuitivo.
- Il frontend dovrà essere **responsive**, garantendo un'esperienza ottimale sia su dispositivi desktop che mobili.
- Un utente istruito deve poter completare un'attività primaria senza necessità di assistenza.
- Un amministratore con formazione adeguata deve poter registrare un nuovo utente in meno di 2 minuti.
- Gli elementi dell'interfaccia e i comportamenti (es. animazioni) devono essere coerenti nelle varie pagine per colori, dimensioni e tempi.

- I nuovi utenti devono poter completare almeno il 90% dei task basilari (creazione, aggiornamento, ricerca, filtraggio, ordinamento delle issues) dopo 30 minuti di addestramento.
- I messaggi di errore devono essere chiari e fornire indicazioni sulle azioni correttive.
- Le funzionalità principali devono essere individuabili senza istruzioni.
- I titoli devono rispettare un limite massimo di caratteri per garantire la leggibilità completa delle issues.
- Ogni funzionalità principale deve essere raggiungibile con un massimo di 5 click dalla homepage.

Efficienza

- Il sistema deve essere performante, con un'esperienza coerente tra versione desktop e mobile.
- Deve essere possibile caricare immagini fino a 5 MB senza compromettere le prestazioni.
- Il tempo di risposta per le operazioni basilari non deve superare 1 secondo.

Affidabilità

- Il sistema deve garantire un **uptime del 99.5%** su base mensile durante gli orari lavorativi.

Sicurezza

- Gli utenti devono cambiare la password temporanea fornita al primo login.
- Le password devono essere sottoposte ad algoritmi di hashing prima del salvataggio.
- Le sessioni devono essere gestite tramite **JSON Web Tokens (JWT)** con validazione.
- Il sistema deve essere protetto da attacchi **CSRF**, **XSS** e **SQL Injection**.
- Le password devono rispettare criteri minimi di complessità (minimo 8 caratteri, almeno una maiuscola, un numero e un carattere speciale).

Manutenibilità

- L'architettura deve essere modulare per facilitare aggiornamenti e manutenzione.
- Deve essere implementato un sistema di **log strutturato** con livelli di severità.
- La documentazione del codice deve seguire gli standard **JavaDoc** (backend) e **JSDoc/TSDoc** (frontend).
- Le dipendenze di terze parti devono poter essere aggiornate senza modifiche strutturali al codice.

2.2.2 Requisiti Organizzativi

Ambientali

- Il sistema dovrà essere ospitato su piattaforme di **cloud computing**, garantendo accesso continuo e universale.
- Il sistema deve essere **containerizzato** tramite **Docker**.
- Deve poter gestire almeno 200 utenti concorrenti senza degrado percepibile delle prestazioni.

Operativi

- Browser supportati: ultime due versioni stabili di **Chrome**, **Firefox**, **Edge** e **Safari**.

Di Sviluppo

- L'intera codebase deve essere versionata tramite **Git**.
- Ogni *pull request* deve essere approvata da almeno un altro sviluppatore.
- La qualità del codice deve essere verificata automaticamente tramite **SonarQube**.
- Gli sviluppatori devono utilizzare strumenti di analisi statica e *linter* integrati nel proprio IDE.

2.3 Modellazione dei Casi d'uso

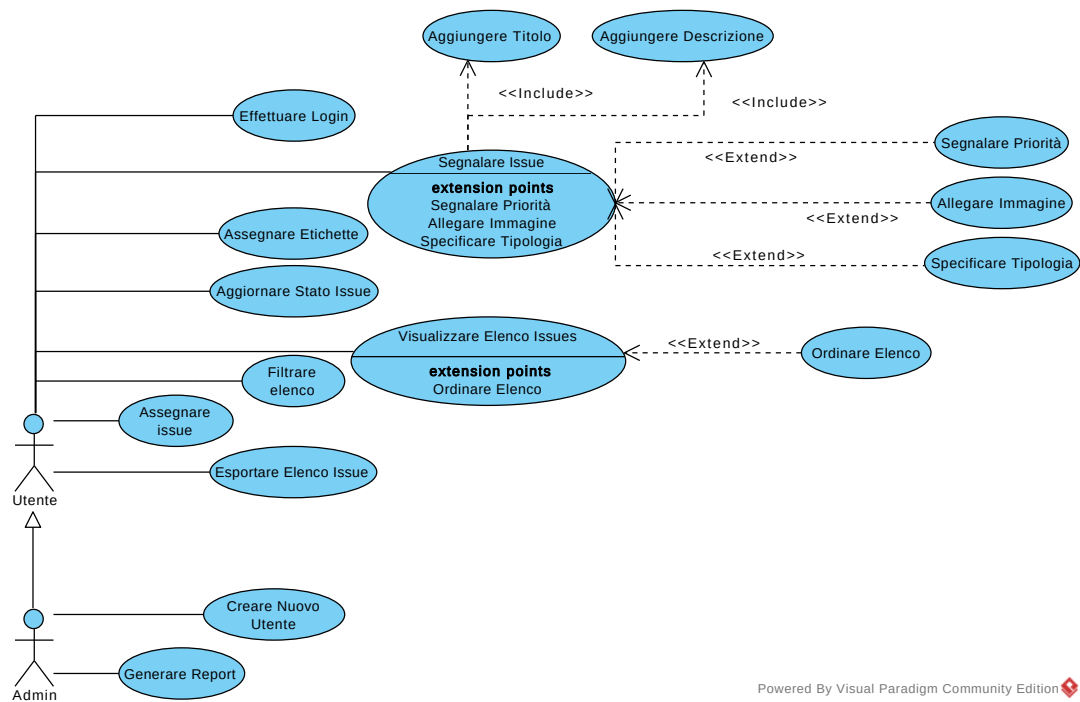


Figura 2.1: Diagramma dei casi d'uso principali di BugBoard26.

2.4 Casi d'uso Significativi

I casi d'uso presentati in questa sezione rappresentano alcune delle interazioni principali che caratterizzano il funzionamento del sistema BugBoard26.

La metodologia di documentazione adottata segue le linee guida contenute in *A Practical Style Guide and Templates Repository for Writing Effective Use Cases* **Wei2022**, che fornisce, citando gli autori, *An opinionated style guide for use case writing* **Washingtonwei2023** attraverso il formalismo di Alistair Cockburn **cockburn-use-cases**. Questo approccio ci consente di descrivere in modo sistematico e coerente:

- L'obiettivo primario di ciascun caso d'uso dal punto di vista dell'attore
- Il flusso principale degli eventi e le relative condizioni di attivazione
- Gli eventuali flussi alternativi
- Le precondizioni necessarie e le postcondizioni attese

Per ogni caso d'uso significativo, forniamo una descrizione strutturata secondo il modello di Cockburn e una tabella delle proprietà rilevanti. Questo livello di dettaglio è necessario per garantire che le funzionalità chiave del sistema siano comprese in modo inequivocabile da tutte le parti interessate, dagli stakeholders agli sviluppatori.

Di seguito riportiamo i casi d'uso selezionati, che costituiscono alcune delle funzionalità *core* di BugBoard26 e rappresentano i principali flussi di interazione con il sistema.

2.4.1 UC1 - Creazione di una issue

ID - Nome dello UC	1 - Creazione di una issue uc1-figma
Data di creazione	11/10/2025
Attore primario	Utente
Trigger	L'utente indica di voler creare una nuova issue
Descrizione	L'utente ha trovato un problema e intende aprire una nuova issue per tenere traccia di esso.

Precondizioni	PRE-1. L'utente è stato registrato da un amministratore ed ha effettuato correttamente l'autenticazione PRE-2. Il sistema ha caricato la pagina di riepilogo delle issues
Postcondizioni	POST-1. La nuova issue è salvata nel sistema POST-2. La nuova issue è visibile nella vista di riepilogo delle issue
Scenario di successo	1. L'utente indica di voler segnalare una nuova issue (M1) 2. Il sistema apre la modale di creazione di una nuova issue (M2) 3. L'utente inserisce il titolo della issue (M3) 4. L'utente indica i dettagli della issue nella descrizione (M4) 4a. [Opzionale] L'utente sceglie una priorità per la issue 4b. [Opzionale] L'utente può allegare un'immagine 4c. [Opzionale] L'utente sceglie un tipo per la issue 5. L'utente segnala di aver terminato l'inserimento dei dati richiesti (M5) 6. Il sistema valida i dati inseriti dall'utente 7. Il sistema salva la nuova issue 8. Il sistema mostra un messaggio di successo (M6)

Continua nella pagina successiva

Estensioni**4a. Scelta della priorità per la issue:**

- 4a1. L'utente clicca sul menù "priorità" (M9)
- 4a2. Il sistema apre un menù con le possibili priorità (M10)
- 4a3. L'utente sceglie una tra le alternative (M11)
- 4a4. Il sistema mostra l'alternativa scelta (M12)
- 4a5. Il flusso ritorna al termine del punto 4 dello scenario di successo

4b. Caricamento di un'immagine:

- 4b1. L'utente clicca sul bottone per caricare un'immagine (M13)
- 4b2. Il sistema apre il file explorer (M14)
- 4b3. L'utente seleziona un'immagine dal proprio dispositivo (M15, M16)
- 4b4. Il sistema valida il formato e la dimensione dell'immagine
- 4b5. Il sistema sostituisce l'icona di "allega" con un messaggio di successo (✓) e aggiunge un bottone di eliminazione accanto alla label "immagine"(M17)
- 4b7. Il flusso torna al termine del punto 4a dello scenario di successo

4b3a. L'utente annulla il caricamento dell'immagine:

- 4b3a1. L'utente clicca sul bottone "annulla"
- 4b3a2. Il flusso ritorna al termine del punto 4 dello scenario di successo

4b4a. Errore di validazione dell'immagine:

- 4b4a1. Il sistema mostra un messaggio di errore e invita l'utente a riprovare (M18)
- 4b4a2. Il flusso ritorna al termine del punto 4 dello scenario di successo

4b6a. Eliminazione dell'immagine allegata:

- 4b6a1. L'utente clicca sul bottone di eliminazione
- 4b6a2. Il sistema elimina l'immagine allegata
- 4b6a3. Il sistema sostituisce l'icona di successo (✓) con il bottone "allega" e rimuove il bottone di eliminazione
- 4b6a4. Il flusso ritorna al termine del punto 4 dello scenario di successo

Continua nella pagina successiva

Estensioni	4c. Scelta del tipo di issue: 4c1. L'utente clicca sul menù "tipo" (M19) 4c2. Il sistema apre il menù con i vari tipi di issues (M20) 4c3. L'utente sceglie un tipo di issue (M21) 4c4. Il sistema mostra il tipo scelto (M22) 4c5. Il flusso ritorna al termine del punto 4 dello scenario di successo 6a. Errore nella validazione dell'input (titolo): 6a1. L'utente inserisce un titolo non valido 6a2. Il sistema evidenzia il campo del titolo e mostra un messaggio all'utente, segnalando la non validità del titolo scelto (M7) 6a3. L'utente modifica il titolo, inserendone uno corretto (M3) 6a4. Il flusso ritorna al punto 4 dello scenario di successo 6b. Errore nella validazione dell'input (descrizione): 6b1. L'utente inserisce una descrizione non valida 6b2. Il sistema evidenzia il campo della descrizione e mostra un messaggio all'utente, segnalando la non validità della descrizione scelta (M8) 6b3. L'utente inserisce una descrizione valida (M4) 6b4. Il flusso ritorna al punto 5 nello scenario di successo 7a. Errore nel salvataggio della issue: 7a1. Il sistema riscontra un errore durante il salvataggio della issue 7a2. Il sistema mostra uno specifico messaggio di errore, invitando l'utente a riprovare (M23) 7a3. Il flusso ritorna al termine del punto 4 nello scenario di successo
Priorità	Alta
Frequenza d'uso	Presumibilmente tutti gli utenti, tra 1 e 5 volte al giorno per utente
Informazioni associate	Mostrato nella tabella ??

Nome della proprietà	Tipo	Regole di validazione
Titolo	String	Obbligatorio, compreso tra 10 e 100 caratteri
Descrizione	String	Obbligatorio, compreso tra 30 e 30000 caratteri
Priorità	String	Facoltativo, valori possibili: - Lowest - Low - Medium - High - Highest
Immagine	File (image)	Facoltativo, dimensione massima: 5Mb
Tipo	String	Facoltativo, valori possibili: - Question - Bug - Documentation - Feature

Tabella 2.4: Regole di validazione delle proprietà

AC1 - Creazione di una issue

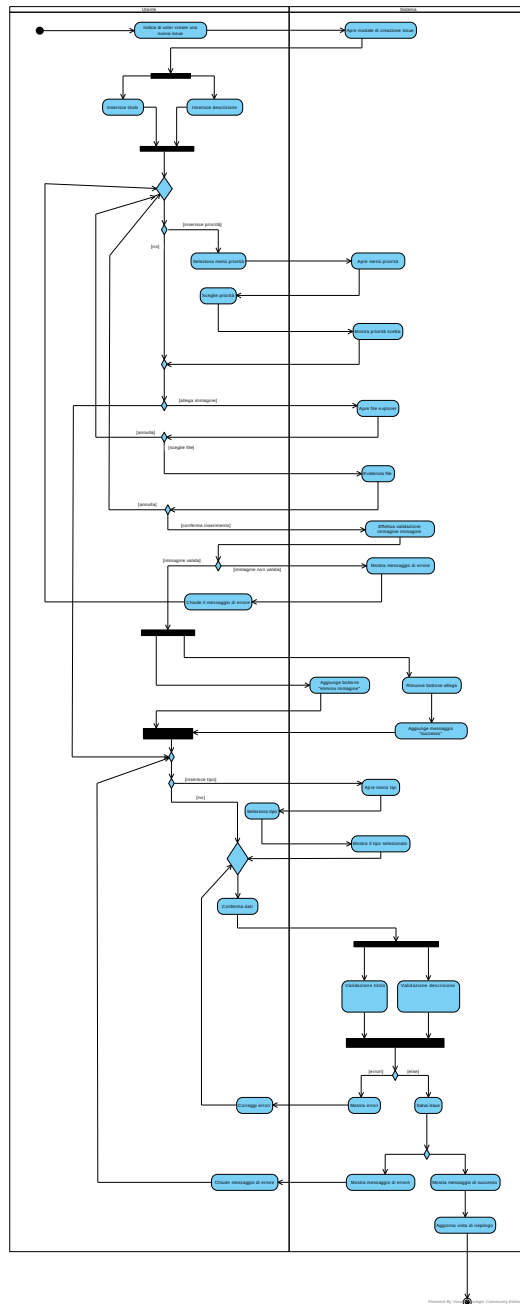


Figura 2.2: Diagramma d'attività della creazione di una nuova issue

2.4.2 UC2 - Associazione di etichette personalizzate

Descrizione Strutturata (Formalismo di A. Cockburn)

Tabella 2.5: UC-2 Associazione Etichette Personalizzate
– Template di Cockburn

Campo	Valore
ID - Nome dello UC	2 - Associazione etichette personalizzate per i Bug uc2-figma
Data di creazione	14/10/2025
Attore primario	Utente
Trigger	L'Utente modifica un bug e indica di voler associare una o più etichette personalizzate
Descrizione	L'Utente desidera associare un numero variabile di etichette personalizzate ad un bug per facilitarne la classificazione.
Precondizioni	• PRE-1. L'Utente è registrato nel sistema. • PRE-2. Il bug selezionato esiste nel sistema.
Postcondizioni	• POST-1. Una o più etichette saranno assegnate al bug scelto dall'utente. Le nuove etichette saranno visibili nella schermata riepilogativa di tutte le issues.
Scenario di successo	1. L'Utente seleziona un bug esistente 2. Il Sistema visualizza i dettagli del bug, comprese eventuali etichette già associate. 3. L'Utente indica di voler aggiungere una nuova etichetta al bug. 4. Il Sistema mostra il pannello per la creazione di un nuovo tag. 5. L'Utente inserisce il nome della nuova etichetta. 6. Il Sistema verifica che il nome inserito rispetti i requisiti 7. Il Sistema verifica che l'etichetta non sia già presente nel sistema 8. L'Utente conferma la creazione dell'etichetta 9. Il Sistema salva le etichette associate e aggiorna la visualizzazione del bug 10. Il sistema notifica l'Utente che le etichette sono state aggiunte con successo

Continua nella pagina successiva

Campo	Valore
Estensioni	6a. Violazione delle regole di validazione 6a1. Il sistema avvisa l'Utente che il nuovo nome inserito non rispetta i requisiti 6a2. L'Utente corregge l'errore, il flusso prosegue normalmente dal passo 5 dello scenario di successo 7a. Etichetta duplicata 7a1. Il sistema rileva che l'etichetta è già presente. 7a2. Il sistema mostra l'etichetta. 7a3. L'Utente seleziona l'etichetta e il flusso prosegue normalmente dal punto numero 8 dello scenario di successo. 7b. Modifica del colore della nuova etichetta 7b1. L'utente seleziona il riquadro per la modifica del colore. 7b2. Il sistema mostra il pannello per la scelta del colore. 7b3. L'Utente seleziona il nuovo colore. 7b4. Il sistema salva la modifica, mostrando il nuovo colore scelto e l'esecuzione procede normalmente dal passo numero 8 dello scenario di successo 9a. Errore nel salvataggio delle nuove etichette 9a1. Il sistema riscontra un errore durante il salvataggio delle nuove etichette associate. 9a2. Il sistema mostra uno specifico messaggio di errore. 9a3. Il flusso ritorna al punto 1 nello scenario di successo.
Priorità	alta
Frequenza d'uso	Alta — tipicamente ogni volta che viene creato o aggiornato un bug.
Informazioni associate	Mostrato nella Tabella ??

Tabella 2.6: UC-2 Associazione Etichette Personalizzate
– Proprietà e Regole di Validazione

Nome della Proprietà	Tipo	Regole di validazione	Note
Nome	String	Obbligatorio, univoco, lunghezza da 1 a 30 caratteri.	L'aggiunta o la modifica aggiornano la descrizione della issue.
Colore	String (HEX)	Facoltativo, formato HEX (RRGGBB), default:808080.	Permette di personalizzare il colore della nuova etichetta, scegliendo tra un set di colori disponibili
Creatore	String	Generata automaticamente	identificativo dell'Utente autenticato che la crea.

2.4.3 UC3 – Ordinamento delle Issue

Descrizione Strutturata (Formalismo di A. Cockburn)

Tabella 2.7: UC-3 Ordinamenti – Template di Cockburn

Campo	Valore
ID - Nome dello UC	3 - Ordinamento uc3-figma
Data di creazione	14/10/2025
Attore primario	Utente
Trigger	L'utente indica di voler ordinare l'elenco delle issues.
Descrizione	L'utente vuole ordinare le issues presenti nell'elenco tramite un criterio specifico.
Precondizioni	• PRE-1. L'utente è stato registrato da un amministratore ed ha effettuato correttamente l'autenticazione. • PRE-2. Il sistema ha caricato la pagina di riepilogo delle issues.

Continua nella pagina successiva

Campo	Valore
Postcondizioni	L'elenco delle issues è riordinato.
Scenario di successo	1. L'utente visualizza la tabella contenente le issues con le relative colonne. 2. L'utente clicca sull'intestazione di una colonna (es. stato, priorità, data creazione) per applicare un ordinamento. 3. Il sistema riordina la tabella in base ai valori della colonna selezionata e mostra un indicatore visivo della direzione dell'ordinamento (es. $\uparrow$ o $\downarrow$). 3. <i>[Opzionale]</i> L'utente clicca su ulteriori intestazioni di colonna per definire un ordinamento secondario o terziario. 4. <i>[Opzionale]</i> Se l'utente clicca nuovamente su una colonna già ordinata, il sistema inverte la direzione dell'ordinamento o ripristina l'ordine predefinito. 5. Il sistema mostra la tabella aggiornata e, se previsto, memorizza i criteri di ordinamento nella sessione o nell'URL.

Continua nella pagina successiva

Campo	Valore
Estensioni	3a. Ordinamento crescente/decrescente 3a1. L'utente clicca una seconda volta sulla stessa intestazione di colonna. 3a2. Il sistema inverte la direzione dell'ordinamento (da crescente a decrescente o viceversa). 3a3. Il sistema aggiorna l'indicatore visivo (freccia $\uparrow / \downarrow$) e ricarica la tabella. 3a4. Il flusso ritorna al termine del punto 3 dello scenario di successo. 4a. Ordinamento secondario o multiplo 4a1. L'utente clicca su un'altra intestazione di colonna diversa da quella attiva. 4a2. Il sistema imposta un secondo livello di ordinamento, mantenendo la gerarchia (es. priority $\rightarrow$ stato). 4a3. Il sistema aggiorna la tabella e mostra indicatori visivi multipli per le colonne coinvolte. 4a4. Il flusso ritorna al termine del punto 4 dello scenario di successo. 4b. Rimozione di un ordinamento attivo 4b1. L'utente clicca nuovamente su una colonna ordinata fino a tornare alla condizione "nessun ordine". 4b2. Il sistema rimuove l'indicatore visivo e ripristina l'ordinamento predefinito. 4b3. Il flusso ritorna al termine del punto 3 dello scenario di successo. 6a. Persistenza dell'ordinamento 6a1. Il sistema salva automaticamente le informazioni sull'ordinamento corrente nella sessione o nell'URL. 6a2. Quando la pagina viene ricaricata o condivisa, l'ordinamento precedente viene ripristinato. 6a3. Il flusso prosegue dal punto 6 dello scenario di successo. 6b. Ordinamento su colonna non ordinabile 6b1. L'utente clicca su una colonna che non supporta l'ordinamento (es. colonna tag). 6b2. Il sistema ignora il click senza modificare la tabella. 6b3. Il flusso ritorna al punto 2 dello scenario di successo.
Priorità	Media

Continua nella pagina successiva

Campo	Valore
Frequenza d'uso	Tutti gli utenti, tra 1 e 3 volte al giorno per utente
Informazioni associate	Mostrato nella Tabella ??

Tabella 2.8: UC-3 Ordinamenti – Proprietà e Regole di Validazione

Nome della Proprietà	Tipo	Regole di validazione	Note
Colonne ordinabili	String	Obbligatorio, click sulla colonna per ordinarlo	Definisce su quali campi è permesso applicare l'ordinamento (nome, <i>stato</i> , <i>priorità</i> , <i>data creazione</i>)
Direzione ordinamento	Boolean (↑/↓)	Invertibile dall'utente	Indica se l'ordinamento è crescente o decrescente
Indicatore visivo	Icona UI	Aggiornato automaticamente	Mostra la direzione e il livello dell'ordinamento accanto all'intestazione
Persistenza ordinamento	URL param	Facoltativa	Se attiva, il sistema salva i criteri nella sessione o nell'URL
Colonne non ordinabili	String	Click ignorato	Colonna Tag

3 Definizione dell'architettura software

3.1 Introduzione

In questa sezione procederemo a definire l'architettura software della piattaforma.

Data la natura dell'applicazione, i casi d'uso individuati e le feature che il sistema dovrà offrire, è stato deciso di sviluppare una web app, accessibile sia da desktop che da mobile (responsive), in maniera da permettere all'intero team di sviluppo di :

- segnalare comodamente i bug da un'interfaccia desktop, inserendo delle descrizioni approfondite
- monitorare lo stato delle diverse segnalazioni, sia da desktop che da mobile, avendo a disposizione molte segnalazioni nella stessa schermata nella versione desktop, ma anche visualizzarne una nel dettaglio tramite l'interfaccia mobile
- garantire la sincronizzazione e la centralizzazione dei dati, indipendentemente dal dispositivo utilizzato per l'accesso

3.2 Architettura Logica

In conformità con i requisiti non funzionali e i vincoli di progetto, l'architettura di *Bug-Board26* è di tipo distribuito, basata sul paradigma Client-Server e sullo stile architetturale *REpresentational State Transfer* (REST).

Il sistema è decomposto in due macro-componenti logicamente e fisicamente separati, che comunicano esclusivamente tramite protocollo HTTP scambiando messaggi in formato JSON:

- **Frontend (Client):** Responsabile della logica di presentazione e dell'interazione utente. Implementato come *Single Page Application* (SPA), questo modulo è completamente disaccoppiato dal backend, garantendo la possibilità di evoluzione indipendente o la sostituzione dell'interfaccia senza impatti sulla logica di business.
- **Backend (Server):** Responsabile della logica di business, della sicurezza e della persistenza dei dati. Il backend espone interfacce di programmazione (API) pubbliche e opera in modalità *stateless*: ogni richiesta contiene tutte le informazioni necessarie per essere elaborata, garantendo la scalabilità orizzontale.

3.3 Pattern Architeturale Backend

Il modulo Backend è strutturato internamente secondo il pattern **Layered Architecture** (Architettura a Strati) per garantire la *Separation of Concerns*. Il flusso di elaborazione di una richiesta attraversa i seguenti livelli:

- **Interface Layer (Controller):** Punto di ingresso delle richieste REST. Gestisce la validazione sintattica degli input (DTO) e la mappatura delle risposte HTTP.
- **Business Logic Layer (Service):** Implementa le regole di dominio, i controlli di sicurezza applicativa e il coordinamento delle transazioni. È isolato dai dettagli dell'interfaccia utente e della persistenza.
- **Data Access Layer (Repository):** Astrae l'accesso al database relazionale tramite un'interfaccia. Fornisce i metodi per la persistenza e il recupero delle entità, disaccoppiando la logica di business dal linguaggio SQL specifico.

3.4 Stack tecnologico backend

Componente	Tecnologia Utilizzata	Motivazione
Linguaggio Backend	Java	Linguaggio fortemente tipizzato, Object-Oriented, utilizzato dal 99% delle aziende java-adoption-rate
Framework Backend	Spring Boot 4	Offre Dependency Injection, semplifica la scrittura del codice rispetto a Java Spring
Database	PostgreSQL	DBMS relazionale robusto, affidabile, sicuro, open source e veloce
Object-Relational Mapping (ORM)	Spring Data JPA	Astrazione necessaria per la gestione della persistenza orientata agli oggetti, riducendo il codice boilerplate e prevenendo nativamente SQL Injection
Container	Docker	Ambiente indipendente che garantisce un funzionamento coerente su ogni ambiente

3.5 Pattern Architetture Frontend

QUESTA PARTE è DA REVISIONARE!!!!!!!!!!!!!!!!!!!!!! Il modulo Frontend è progettato come una *Single Page Application* (SPA) basata su una **Component-Based Architecture**. L'interfaccia utente è decomposta in unità funzionali indipendenti e riutilizzabili (Componenti), organizzate in una struttura gerarchica ad albero.

Per garantire la manutenibilità e la testabilità del codice client, viene applicata la separazione tra:

- **Container Components (Smart):** Responsabili del recupero dei dati, della gestione dello stato della vista e dell'interazione con i Service di comunicazione.
- **Presentational Components (Dumb):** Responsabili esclusivamente del rendering grafico e dell'intercettazione degli eventi utente, privi di dipendenze dalla logica di business o dai servizi esterni.

Gestione della Comunicazione Asincrona L'interazione con le API REST del Backend è gestita secondo il **Observer Pattern**. Utilizzando la libreria *RxJS*, le risposte HTTP sono trattate come flussi di dati (*Streams*) asincroni. Questo approccio reattivo permette all'interfaccia di aggiornarsi dinamicamente al variare dello stato dei dati senza bloccare l'interazione utente, migliorando la percezione di fluidità e performance del sistema.

3.6 Dettaglio Tecnologie Frontend

A integrazione della Tabella ??, si motivano le scelte specifiche per il livello di presentazione:

- **Framework: Angular (v17+).** La scelta è motivata dalla natura *strongly-typed* offerta da TypeScript, che garantisce coerenza con il backend Java e riduce gli errori a runtime. Il sistema di *Dependency Injection* nativo di Angular facilita il testing unitario dei componenti e dei servizi di comunicazione.
- **Styling: SCSS e Angular Material.** L'adozione di SCSS permette l'uso di variabili e mixin per mantenere un design system coerente. La libreria di componenti *Angular Material* assicura il rispetto delle linee guida di usabilità e accessibilità (WCAG) con un minimo sforzo implementativo.

3.7 Architettura Fisica e Deployment

L'infrastruttura di **BugBoard26** è interamente ospitata su una piattaforma di cloud computing (come, ad esempio, AWS) per garantire un accesso continuo e universale al sistema. Il deployment si basa sulla containerizzazione tramite **Docker**, che fornisce un ambiente di esecuzione isolato e indipendente, garantendo un funzionamento coerente dell'applicativo su qualsiasi ambiente hardware o virtuale sottostante.

Fisicamente, il deployment si articola nei seguenti nodi e ambienti di esecuzione:

Nodo Cloud - Application Server (Backend): All'interno dell'infrastruttura cloud, uno o più container Docker sono dedicati all'esecuzione del modulo applicativo sviluppato in Java e basato sul framework *Spring Boot 4*.

Nodo Cloud - Database Server: Un container Docker ospita il DBMS relazionale *PostgreSQL*, impiegato per la persistenza dei dati. Il backend comunica fisicamente con questo nodo sfruttando l'astrazione ORM fornita da *Spring Data JPA*.

Nodi Client (Dispositivi Utente): L'accesso al sistema avviene tramite i dispositivi hardware fisici (desktop o mobile) utilizzati dal team. L'ambiente di esecuzione su questi nodi è costituito da un browser web. Sul browser viene scaricata ed eseguita l'interfaccia grafica sviluppata in *Angular* e stilizzata tramite SCSS e *Angular Material*.

3.8 Descrizione dello schema per la persistenza dati

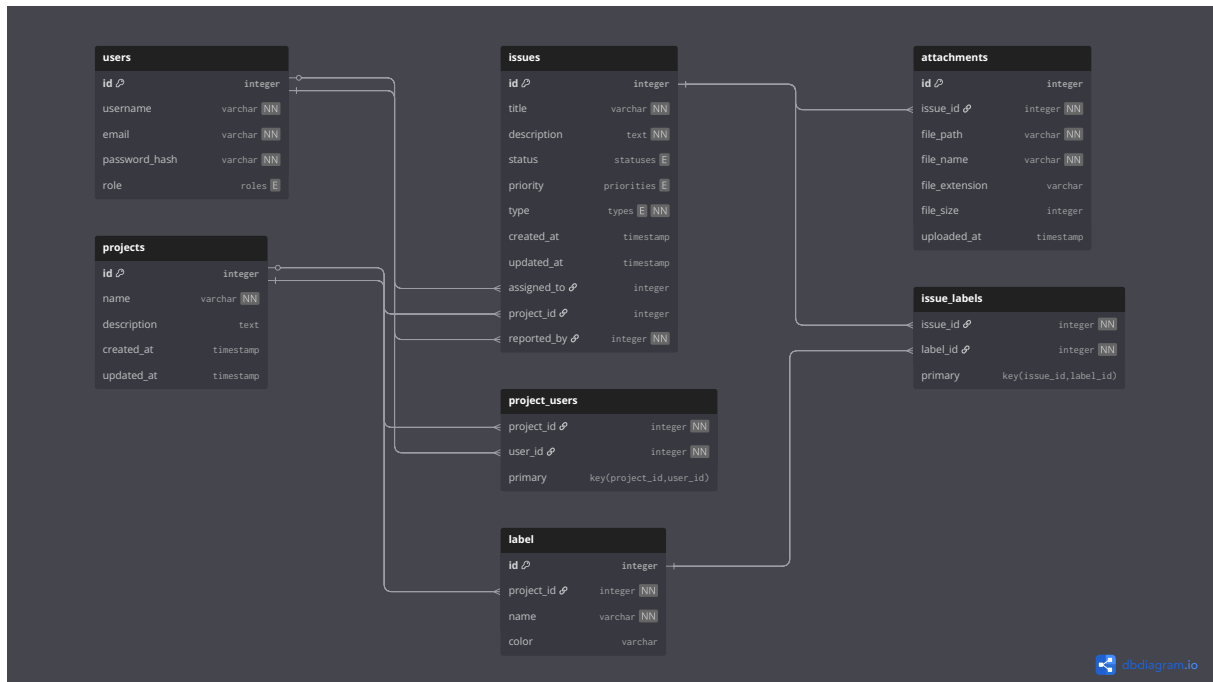


Figura 3.1: Schema di persistenza dei dati

4 Definizione dei Design Patterns

L'architettura del sistema si avvale dell'utilizzo di design pattern consolidati per garantire la manutenibilità, l'estensibilità e la robustezza del codice. L'adozione di tali soluzioni permette di separare le responsabilità dei componenti, facilitando l'evoluzione del software in risposta a nuovi requisiti senza compromettere le funzionalità esistenti. Di seguito vengono analizzati i pattern principali implementati nel progetto.

4.1 Singleton Pattern

Per l'accesso alla base dati e la gestione delle connessioni a PostgreSQL, l'architettura sfrutta implicitamente il Singleton Pattern per garantire un accesso centralizzato e ottimizzato. Tuttavia, in aderenza alle best practice del framework Spring Boot, si è evitato di implementare manualmente il classico pattern GoF (con costruttore privato e metodo statico). Si è scelto invece di demandare il ciclo di vita dei componenti al framework, sfruttando la Dependency Injection. I Repository di Spring Data JPA (es. IssueRepository, UserRepository) sono generati e gestiti dal container Inversion of Control di Spring con scope Singleton di default. Questo approccio unisce i vantaggi prestazionali del Singleton (un solo gestore del Connection Pool in memoria) a quelli del basso accoppiamento garantito dalla Dependency Injection, rendendo inoltre le classi facilmente testabili in isolamento.

4.2 Dependency Injection e Inversion of Control (IoC)

La Dependency Injection rappresenta il cardine architettonico dell'intero sistema. Invece di permettere ai componenti di istanziare autonomamente le proprie dipendenze (creando un forte accoppiamento), il controllo viene delegato al container di Spring Boot. L'implementazione segue il principio dell'*Injection tramite costruttore*:

- Il `IssueController` riceve un'istanza di `IssueService`.
- Il `IssueService` riceve i repository necessari (`IssueRepository`, `TagRepository`).

Questo approccio garantisce un **basso accoppiamento** e un'alta testabilità: è infatti possibile iniettare *Mock Objects* durante i test di unità per isolare la logica di business senza interagire con il database reale. Inoltre, la DI facilita l'implementazione dello *Strategy Pattern* precedentemente descritto, permettendo al framework di selezionare dinamicamente l'implementazione corretta della strategia di esportazione al momento dell'esecuzione.

4.3 State Pattern

Implementato per gestire il ciclo di vita delle segnalazioni (*Issue*). Dato che una segnalazione può attraversare molteplici stati (es. `ToBeAssigned`, `InProgress`, `MarkedForReview`) con logiche di transizione complesse, l'uso di semplici costrutti condizionali avrebbe reso il codice fragile e difficile da mantenere. L'interfaccia `Status` definisce il contratto per tutte le possibili azioni, mentre la classe astratta `BaseStatus` fornisce un'implementazione di default (tipicamente inibendo le azioni non permesse). Ogni classe concreta rappresenta uno stato specifico e incapsula la logica delle transizioni ammesse. Questo permette alla classe `Issue` di delegare il comportamento dinamico all'oggetto `status` corrente, semplificando l'aggiunta di nuovi stati senza modificare la logica di business esistente.

4.4 Strategy Pattern

Utilizzato per la gestione dell'esportazione dei dati delle *Issue* in diversi formati (es. CSV, JSON). Attraverso l'interfaccia `IssueExportStrategy`, il sistema definisce una famiglia di algoritmi di esportazione intercambiabili. Il servizio `IssueService` agisce come contesto, mantenendo un riferimento alla strategia scelta ma rimanendo agnostico rispetto ai dettagli implementativi della stessa. Questa separazione consente di aggiungere nuovi formati di esportazione semplicemente implementando una nuova classe concreta, rispettando il principio *Open/Closed*: il sistema è aperto all'estensione (nuovi formati) ma chiuso alla modifica (il codice del servizio non cambia).

5 Glossario

A

Admin utente con privilegi di gestione, in grado di creare e assegnare utenti e issue.

Affidabilità misura della capacità del sistema di funzionare in modo corretto e continuativo nel tempo.

Angular framework JavaScript per lo sviluppo di applicazioni web single-page.

Architettura struttura logica e fisica che definisce l'organizzazione dei componenti software.

B

Backend parte dell'applicazione che gestisce la logica, il database e la comunicazione tra client e server.

Background tecnico insieme delle conoscenze, competenze e tecnologie pregresse che costituiscono la base teorica e pratica di un team di sviluppo.

Boolean tipo di dato che può assumere solo due valori: vero o falso.

Bug errore o comportamento anomalo nel software che causa un malfunzionamento.

C

Casi d'uso rappresentazione delle interazioni tra un attore e il sistema per raggiungere un obiettivo specifico.

Chrome browser web sviluppato da Google.

Click azione dell'utente che interagisce con un elemento grafico.

Cloud computing modello di erogazione di servizi informatici tramite Internet.

Code quality (Qualità del codice) grado di leggibilità, modularità e manutenibilità del codice sorgente.

Commenti annotazioni nel codice che descrivono funzionalità o scelte implementative.

Concorrenti utenti o processi che operano simultaneamente sul sistema.

Containerizzato modalità di distribuzione del software all'interno di contenitori isolati, ad esempio tramite Docker.

CSRF vulnerabilità che consente ad un utente malintenzionato di eseguire azioni non autorizzate su un sito autenticato.

D

Deadline scadenza entro la quale una determinata attività o task deve essere completata.

Degrado peggioramento delle prestazioni del sistema dovuto a carichi elevati.

Dipendenze librerie o componenti esterni necessari al corretto funzionamento del software.

Docker piattaforma per creare, distribuire ed eseguire container software.

E

Edge browser web sviluppato da Microsoft.

Efficienza misura delle risorse necessarie per eseguire operazioni (tempo, memoria, CPU).

Esportare operazione per convertire o salvare dati in formati condivisibili (CSV, PDF, ecc.).

Etichette parole chiave che classificano o raggruppano issue e bug.

F

File entità che rappresenta un'informazione salvata in memoria.

File explorer finestra di sistema usata per cercare e selezionare file locali.

Firefox browser open source sviluppato da Mozilla.

Flusso sequenza ordinata di azioni o eventi.

Formalismo insieme delle regole che definiscono il modo corretto di rappresentare i casi d'uso.

Framework struttura software riutilizzabile che facilita lo sviluppo applicativo.

Frontend parte dell'applicazione visibile e interattiva con l'utente.

Funzionalità core insieme delle funzioni principali del sistema.

G

Gerarchia relazione di subordinazione o dipendenza tra elementi o ruoli.

Git sistema di controllo versione distribuito.

H

Hashing tecnica per trasformare dati (come password) in una stringa di lunghezza fissa non invertibile.

HEX / RRGGBB codifica esadecimale utilizzata per rappresentare i colori.

Hibernate framework Java per la gestione della persistenza dei dati.

I

Icona elemento grafico che rappresenta un'azione o funzione.

IDE ambiente di sviluppo integrato che facilita la scrittura e il debug del codice.

Identificativo codice univoco che distingue un elemento nel sistema.

Image file grafico o rappresentazione visiva associata a un'issue.

Interfaccia insieme degli elementi grafici e funzionali con cui l'utente interagisce.

Interazioni comunicazioni o scambi di informazioni tra attori e sistema.

Issue registrazione di un problema, richiesta o attività da gestire nel sistema.

J

Jakarta EE piattaforma Java per lo sviluppo di applicazioni enterprise.

Java linguaggio di programmazione orientato agli oggetti.

JavaDoc sistema di documentazione automatica per codice Java.

JSON Web Token (JWT) standard aperto per la trasmissione sicura di informazioni tra client e server.

JS Doc strumento per generare documentazione dal codice JavaScript.

L

Lint strumento che analizza il codice per rilevare errori sintattici o stilistici.

Linguaggio di programmazione mezzo formale per scrivere istruzioni comprensibili da un computer.

Login procedura di autenticazione che consente l'accesso al sistema.

Log strutturato registro di eventi con formato standard che facilita l'analisi automatizzata.

M

Manutenibilità facilità con cui il software può essere corretto o migliorato nel tempo.

MB unità di misura della quantità di dati (megabyte).

Menù elenco di opzioni tra cui l'utente può scegliere.

Messaggio di errore testo mostrato quando si verifica un problema nell'esecuzione.

Metodologia insieme di processi e strumenti adottati nello sviluppo software.

Metodologie di sviluppo approcci sistematici come Agile, Waterfall.

Modale finestra sovrapposta che richiede l'interazione dell'utente.

Modello di Cockburn formalismo per la descrizione strutturata dei casi d'uso.

Modifiche strutturali cambiamenti significativi nell'architettura o nella base di codice.

O

Requisiti operativi requisiti che definiscono le condizioni di utilizzo e manutenzione del sistema.

U

UI-(User Interface) interfaccia grafica con cui l'utente interagisce.

URL indirizzo che identifica una risorsa in rete.

URL param parametro aggiunto a un URL per trasmettere informazioni.

Usabilità facilità con cui l'utente può utilizzare un sistema.

Up time tempo in cui il sistema risulta operativo e accessibile.

V

Validazione processo con cui si verifica che un sistema, prodotto o software soddisfi realmente i bisogni e le aspettative dell'utente finale.

Version Control sistema che gestisce e traccia le modifiche apportate ai file di un progetto nel tempo.

X

XSS Vulnerabilità nelle applicazioni web che permette a un attaccante di inserire ed eseguire script non autorizzati nel browser di un altro utente